`tree-dp`: the preference-forest dynamic program for differential compression of the gapped suffix array

Abstract

`tree-dp` compresses a gapped suffix array by choosing, for each gapped k-mer name, at most one *link* that replaces a run of stored positions by a pointer plus a shift. It does so in five stages: enumerate a pruned candidate universe; let every name nominate a single *preferred* link; observe that the resulting "who sources from whom" graph has out-degree at most one and is therefore a *pseudoforest*; delete one edge per cycle to obtain a forest; and solve the restricted problem on that forest *exactly* by a two-state Keep/Compress dynamic program. A materialization pass, an availability greedy over the names the forest never reached, and a shared unpin/retarget fixed point complete the algorithm. This note is the definitive description of that base algorithm. It states and proves the pseudoforest property that makes the DP possible; it isolates the three places where optimality is lost (one link per name, one parent per name, and an arbitrarily chosen cut edge); and it demonstrates the third loss on a family of 24 instances that are pairwise *isomorphic* as optimization problems — identical interval sizes and identical 15-link universes under a relabelling, and optimum $|C| = 11$ throughout — on which `tree-dp` returns 11 for eight of them and 13 for the other sixteen, while every order-independent algorithm in the family returns one value for all 24. The difference is nothing but the arithmetic order of four k-mer names. Everything numerical here was produced by the committed binaries at `7f9127b`.

Contents

1 Scope, and how this note relates to the others **2**

2 Setup **3**

3 Phase 0: candidate enumeration **4**
3.1 The scan . . . 4
3.2 Availability . . . 4
3.3 Intra-interval links . . . 5
3.4 The role of `-max-add` . . . 5

4 Automatic $a_{\max}$ and the wrap-around add **5**
4.1 When a small $a_{\max}$ cannot wrap . . . 5
4.2 Predicting P from the GSA . . . 6
4.3 The coverage-desc cache must not evict $a^\star$ . . . 6
4.4 What this does, and what it does not . . . 7

5 Preference selection and the pseudoforest **8**
5.1 Preferences . . . 8
5.2 The preference graph . . . 8

6 Cycle breaking **9**

7 The KEEP/COMPRESS dynamic program **10**
7.1 States and recurrence . . . 10
7.2 The root rule . . . 10
7.3 Intra-interval links are invisible to the DP . . . 10
7.4 Exactness on the forest . . . 11
7.5 Memoization and parallelism . . . 11

8 Materialization: the accept pass and the leftover greedy **12**
8.1 Accepting deepest-first . . . 12
8.2 The leftover greedy . . . 12

9 Phase II: the unpin/retarget loop **13**
9.1 The retarget move . . . 13
9.2 The dirty-set work queue . . . 13
9.3 The budget . . . 14
9.4 Why Phase II is invisible on small instances . . . 14

10 The algorithm in full **15**

11 Worked example: `(AGCT)`^{5} with shape `#.#` **15**
11.1 Why this instance . . . 15
11.2 The instance . . . 16
11.3 Phase 0: the candidates . . . 16
11.4 Phase 1: the preference graph . . . 17
11.5 Phase 1: cycle breaking . . . 17
11.6 Phase 1: the DP . . . 18
11.7 Phase 1: accept and leftover . . . 18
11.8 Phase II and the result . . . 19

12 The isomorphic twin: where cycle breaking costs two positions **20**
12.1 The two instances are the same problem . . . 20
12.2 What differs . . . 20
12.3 Why the path loses . . . 20
12.4 Two restrictions, interacting . . . 21

13 The smallest counterexample, and what no later pass can fix **22**

14 Continuity: the walkthrough's running example **23**

15 Complexity and practical behaviour **23**
15.1 Asymptotics . . . 23
15.2 Measured phase timings . . . 24
15.3 Parallel speedup . . . 24

16 Where `tree-dp` stands **24**
16.1 Small instances against the ILP optimum . . . 24
16.2 The 139-configuration suite . . . 24
16.3 Summary of the empirical picture . . . 25

17 Discussion **25**

1 Scope, and how this note relates to the others

Four documents in `doc/` now describe overlapping parts of the same pipeline.

- `dep_order_walkthrough` builds the gapped suffix array from scratch on a three-fold repeat of `GCCTTTAAAG`, defines links, and walks through `greedy`, `dep-order` and (in §6) an early version of `tree-dp`. It is the right place to learn what a gapped suffix array *is*.
- `tree_dp3` formalizes the optimization problem, proves it is a set-packing problem with two *pairwise* constraints and no acyclicity requirement, and builds the exact cluster-local re-optimization `tree-dp3` on top of that.
- `compact_ilp` writes the set-packing ILP that `ilp_baseline` emits, then the equivalent compact form that decides one consecutive sub-run per maximal source run instead of one binary per sub-run — $O(m\, a_{\max})$ binaries rather than $\Theta(\sum L^2)$. That is the formulation to solve on tandem inputs where the expanded full universe will not even write.
- This note treats `tree-dp` itself in full.

We take the notation of `tree_dp3` §1 as given: a link is a tuple $k = (c_k, a_k, S_k, T_k)$ with target name c_k, shift a_k, source ranks S_k and covered ranks T_k; validity conditions (V1)–(V3); feasibility conditions (F1) "one link per name" and (F2) "a chosen link's source ranks are all kept"; and the objective

$$|C|(X) \;=\; m - \mathrm{cov}(X), \qquad \mathrm{cov}(X) = \sum_{k \in X} \mathrm{cov}(k), \qquad \mathrm{cov}(k) = |T_k| = |S_k|.$$

Section 2 recalls just enough of this to be readable standalone. Where `tree_dp3` already proves something we cite it rather than repeat it.

The reader who wants only the algorithm can read §3–§9; the reader who wants to see it run should go to §11.

A note on provenance. All measurements below come from a clean build of the tree at commit `7f9127b` (*Add the tree-dp3 writeup*), extracted with `git archive` into a scratch directory and built with the repository `Makefile`. This matters because the working tree carried unrelated uncommitted changes at the time of writing.

2 Setup

Fix a binary shape S of span s over a text $T[0..n)$. Every position p carries a gapped k-mer *name*, the w characters selected by the care positions of S placed at p. The DisLex transform groups the names by residue class $p \bmod s$ and the suffix array of the resulting lextext is the **gapped suffix array** (GSA), with $m = n + 1$ ranks. We write $\mathrm{orig}(r)$ for the text position of rank r and $\mathrm{name}(r)$ for its gapped k-mer.

Because the GSA is sorted by gapped suffix, the ranks carrying a fixed name c are contiguous: they form the **interval** $I_c = [\ell_c, r_c)$ of size $\mathrm{isize}(c) = r_c - \ell_c$. The intervals partition $\{0, \ldots, m-1\}$. Let $\mathcal{N}$ be the set of names with $\mathrm{isize}(c) > 1$; singleton intervals are never compressed.

Definition 1 (Link)**.** A **link** k with target name c, shift $a \geq 1$, source run $S_k = [s_{\mathrm{lo}}, s_{\mathrm{hi}})$ and covered set T_k is *valid* when

(V1) shifting the j-th source rank by a windows gives the j-th covered rank, that is $\mathrm{orig}(T_k[j]) = \mathrm{orig}(S_k[j]) + a \cdot s$, and every covered rank carries the name c;

(V2) $S_k \cap T_k = \emptyset$;

(V3) $\mathrm{cov}(k) = |T_k| = |S_k| \geq \kappa$, where $\kappa = 3$ is `kMinCoverage`.

Accepting a link means: drop every rank of T_k from the compressed array C, and store in the hash entry for c an $H_{\text{offset}} = (pos, a, num)$ recording where S_k starts in C, the shift, and how many positions it regenerates. The ranks of I_c that no link covers are stored literally and pointed at by H_{rest}. Decoding is a single addition per covered position and never recurses — a fact that `tree_dp3` (Proposition 2) shows removes any acyclicity requirement from the problem, and that we will lean on in §9.

A set X of valid links is **feasible** when every pair $k \neq k'$ in X satisfies

(F1) $c_k \neq c_{k'}$ (one H_{offset} per hash entry), and

(F2) $S_k \cap T_{k'} = \emptyset$ (a source must be physically present in C).

Since $|C|(X) = m - \mathrm{cov}(X)$, minimizing $|C|$ is maximizing total coverage subject to (F1) and (F2). Throughout, $a_{\max}$ denotes `-max-add` (default 8).

3 Phase 0: candidate enumeration

The link universe of §2 is far too large to hand to a heuristic: it has $\Theta(m \cdot a_{\max})$ maximal runs and, if sub-runs are counted, $\Theta(\sum_R L_R^2)$ links (up to $\Theta(a_{\max} m^2)$). `enumerate_candidates_` produces a pruned view of it, one interval at a time. This pruned view is what every heuristic sees; the full universe is enumerated only by `enumerate_all_links_`, which feeds the expanded ILP baseline and therefore gives a genuine lower bound on all of them. On tandem inputs that expanded LP does not fit; the equivalent compact formulation (one occupancy mask per maximal run, $O(m\, a_{\max})$ binaries) is in `compact_ilp`.

3.1 The scan

Fix a name c with interval $I_c = [\ell, r)$ and a shift a. For each rank $t \in I_c$ the routine asks: is there a rank u whose text position is exactly $\mathrm{orig}(t) - a \cdot s$, in the same residue class? This is `pred_lexpos_`, a constant-time lookup in the lextext. The surviving pairs (u, t) are sorted by u, giving a list

$$pr = \big[(u_1, t_1), (u_2, t_2), \ldots\big], \qquad u_1 < u_2 < \cdots .$$

Each pair is a one-position link; a candidate is a *run* of consecutive pairs.

The run is cut at two kinds of boundary. The first is obvious: u must advance by exactly one rank, $u_j = u_{j-1} + 1$. The second is the interesting one:

$$\mathrm{lcp}[u_j] \;\geq\; a + 1.$$

Remark 1 (Why $\mathrm{lcp} \geq a+1$). Adjacent ranks u_{j-1}, u_j agreeing in their first $a+1$ gapped symbols is exactly the condition under which shifting both forward by a windows lands them in the *same* target interval and preserves their relative order. Weaker agreement can still yield valid links — `enumerate_all_links_` emits them — but the resulting source runs are not lcp-intervals and the enumeration loses the property that each run is described by a single contiguous rank range. The heuristics accept the loss: $\mathrm{lcp} \geq a + 1$ is a pruning rule, not a correctness one.

A run of length cov shorter than $\kappa = 3$ is discarded outright (`emit_run_` returns immediately). This floor is not a tuning parameter that happens to be set to 3; it is the point below which a link stops paying for itself, because the hash entry that stores it costs space too.

Remark 2 (The floor is visible in the older walkthrough). On the walkthrough's running example $T = (\texttt{GCCTTTAAAG})^3$ with $S = \#.\#$, the names `CT` and `GC` were described as preferring each other with coverage 2. At `7f9127b` those links are never enumerated at all, and `CT` and `GC` have *no* candidates whatsoever. See §14.

3.2 Availability

`enumerate_candidates_` takes a flag `require_avail`. When set, a candidate is emitted only if

- every rank of its source run is still kept (`run_kept_`), which is (F2); and
- no rank it covers is already removed, and no rank it covers is *pinned* — i.e. is currently serving as a source rank for some accepted link.

The pin counter `pin_count_` is what enforces (F2) dynamically. Preferences (§5) are computed with `require_avail=false`: they are a static, structural notion, computed once before anything has been accepted, and cached for reuse by the leftover pass and Phase II.

3.3 Intra-interval links

Historically a source run overlapping I_c was rejected: a link was required to source from a *different* name's interval. That rule is stronger than necessary. What (V2) actually demands is only $S_k \cap T_k = \emptyset$: the source must not contain any of the ranks the link itself deletes. A run that overlaps I_c but avoids its own covered ranks decodes perfectly well.

When the source run of the scan lands inside I_c, `emit_intra_runs_` therefore does not discard it; it splits it into maximal sub-runs that are free of the self-conflict. In run-relative coordinates, position t of the run covers rank u; the window $[p, q)$ is bad exactly when it contains both t and u for some pair. The routine precomputes, for each right endpoint b, the smallest window start that still traps a pair ending at b, then sweeps q with a monotone left pointer p, emitting a window whenever it cannot be grown to the right without moving p. Monotonicity of p means no emitted window contains another, so the output is the antichain of maximal conflict-free sub-runs, computed in linear time in the run length.

The sub-run splitting is what makes intra-interval links useful rather than merely legal: the whole run is almost always self-conflicting, while its halves are not.

Example 1 (Intra-interval links are not a corner case). Take $T = \mathtt{A}^{16}$, $S = \#.\#$, so $m = 17$. There are three names: \$\$ (one rank), A\$ (two ranks) and AA (fourteen ranks, $I_{\mathtt{AA}} = [3, 17)$). Only AA is compressible, and the only place a source run can possibly come from is $I_{\mathtt{AA}}$ itself. Then

$$\texttt{tree-dp} \Rightarrow |C| = 11, \qquad \texttt{GCSA_INTRA_LINKS=0} \Rightarrow |C| = 17.$$

With intra-interval links disabled the instance is incompressible; with them enabled `tree-dp` attains $|C| = 11$, which the ILP baseline confirms is optimal. This example returns in §7.3, where it exposes something less comfortable.

3.4 The role of `-max-add`

The scan runs $a = 1, \ldots, a_{\max}$. Raising $a_{\max}$ strictly enlarges every candidate list, so it can only help a hypothetical optimizer — and, empirically, it does help the exact one: at $a_{\max} = 16$, `tree-dp3` improves suite-wide $|C|$ by nearly 16 % over $a_{\max} = 8$, whereas `tree-dp` barely moves (§16). For `tree-dp` the extra candidates are mostly invisible, because each name still contributes exactly one preferred link to the forest and the preferred link rarely changes: a larger shift is chosen only to break a coverage tie. The candidate universe is not `tree-dp`'s binding constraint; the forest is.

On a tandem, however, a *specific* large a is not a coverage tie-break. It is the only add that maps one copy of a k-mer onto another copy of the same k-mer. The rest of this section is the mechanism that finds that add from the GSA and then actually keeps it in the candidate lists that every heuristic, including `tree-dp`, reads.

4 Automatic $a_{\max}$ and the wrap-around add

4.1 When a small $a_{\max}$ cannot wrap

A link with add a recovers a covered rank t from a source rank u by the original-text shift

$$\mathrm{orig}(t) \;=\; \mathrm{orig}(u) + a \cdot s$$

(§2, (V1)). On a tandem of period P — x identical copies of a motif of length P — the occurrences of any fixed k-mer sit at original positions x_0, x_0+P, x_0+2P, ... The shift $a \cdot s$ lands on another occurrence of the *same* k-mer if and only if

$$a \cdot s \;\equiv\; 0 \pmod{P}, \tag{1}$$

i.e. iff a is a multiple of

$$a^\star \;=\; \frac{P}{\gcd(P, s)}. \tag{2}$$

This is the **wrap-around add**. For the contiguous shape of weight 21 used below, $s = 21$. On a motif of length 50, $\gcd(50, 21) = 1$ so $a^\star = 50$; on length 100, $a^\star = 100$; on length 200, $a^\star = 200$. None of these is visible at the default $a_{\max} = 8$. The name-cycle packing that $a \le 8$ *can* see — hopping $a \cdot s \bmod P$ positions along the motif, keeping a dominating set of names as sources — is a different, and on these inputs strictly coarser, feasible set: at $a_{\max} = 8$ the maximal universe ILP on the 200×50 tandem is $|C| = 1285$, against 786 once $a^\star = 50$ is admitted.

4.2 Predicting P from the GSA

P is not an input. After the GSA is built, and before any candidate is enumerated, `predict_max_add_` reads it off the original positions already stored with the ranks. The procedure is Algorithm 1; it is computed once per build and then used by every compressor (`tree-dp`, greedy-degree, dep-order, and the ILP baseline).

Algorithm 1 Predict $a_{\max}$ from orig-position gaps (`predict_max_add_`)

Input: gapped suffix array G of length n, intervals $\{I_c\}$, stride s, floor $a_0 = 8$, cap $A = 65536$
Output: $a_{\max}$

$H \leftarrow$ empty multiset of gaps; *maxgap* $\leftarrow \max(4096,\, n/4)$
for all names c with $\mathrm{isize}(c) \ge \kappa$ **do**
  let $p_1 < \cdots < p_{\mathrm{isize}(c)}$ be $\{\mathrm{orig}(r) : r \in I_c\}$ sorted
  for $i = 2$ **to** $\mathrm{isize}(c)$ **do**
    if $0 < p_i - p_{i-1} \le$ *maxgap* **then**
      insert $p_i - p_{i-1}$ into H
    end if
  end for
end for
$P \leftarrow$ the mode of H (ties to the smaller gap)
if $|H| = 0$ **or** the multiplicity of P is < 8 **or** that multiplicity is $< |H|/4$ **then**
  return a_0 ▷ no spike: random DNA, unique k-mers
end if
return $\min\big(A,\, \max(a_0,\, P/\gcd(P, s))\big)$

Two filters stop the mode from being a pangenome artefact. Gaps larger than $\max(4096, n/4)$ are discarded, so a $\sim$ 5–10 Mbp inter-genome offset is not counted as a tandem period on a 50 Mbp concatenation. Independently, the mode must be a *spike*: at least eight pairs, and at least a quarter of all counted gaps. On the 10-strain E. coli concatenation the mode is $P = 10\,438\,234$

(the spacing of homologous k-mers across genomes) but it is only 9.2 % of the gaps, so the test refuses it and $a_{\max}$ stays 8. On every 100 %, 50 % and 10 % file of the small repeat suite the gaps are identically P and the test accepts; `-max-add auto` then prints $a_{\max} \in \{50, 100, 200\}$ according to the motif length.

`-max-add auto` is the CLI spelling; internally $a_{\max} \leq 0$ is the sentinel that triggers Algorithm 1 after the GSA exists, because the histogram is a function of orig and the intervals, not of the raw text.

4.3 The coverage-desc cache must not evict $a^\star$

Enumeration does not hand every legal link to the heuristics. Each name keeps a cache of at most Γ candidates, sorted by (cov $\downarrow$, $a \downarrow$) — the same order `pick_best_` uses — and `emit_run_` refuses to materialize anything that cannot beat the current worst retained entry. The default was $\Gamma = 64$ (`kCandCacheDefaultCap`). That constant is invisible at $a_{\max} = 8$. It is not invisible at $a_{\max} = a^\star$.

On the 100×100 tandem, auto sets $a_{\max} = 100$. The wrap adds $a \in \{99, 100\}$ have, individually, lower coverage than a band of mid-range adds $a \in \{64, \ldots, 68\}$ (coverage 86–87 versus the wrap's fragmented intra-runs). Coverage-desc eviction therefore retained the mid-range band and dropped $a^\star$. Greedy-degree accepted adds 64–68, Phase II could not invent the wrap from a cache that did not contain it, and $|C|$ was 1741 — *worse* than $a_{\max} = 8$ (1645). Raising Γ to 128 lets $a^\star$ survive; Phase II then collapses the same Phase I packing to $|C| = 999$. The 200×50 tandem never saw the bug: $a^\star = 50 < 64$, so wrap was already in the cache and $|C| = 786$ with or without the change.

The live cap is therefore

$$\Gamma \;=\; \max(64,\, a_{\max}), \tag{3}$$

unless `GCSA_CAND_CACHE_CAP` is set, in which case that value is used exactly (a diagnostic override, not a second default). On E. coli auto keeps $a_{\max} = 8$, so (3) is a no-op and the cache does not grow.

4.4 What this does, and what it does not

Table 1 is greedy-degree, weight-21 contiguous, `-max-add auto`, $\Gamma = \max(64, a_{\max})$, on the seed-1 repeat suite of `bench/datasets.py`. Best-cov is $m - \sum_c \max_{k \in \hat{\mathcal{K}}_c} \mathrm{cov}(k)$, conflicts ignored — a lower bound on *this* search, not on the true optimum. On the three 100 % files it equals 42; the feasible floor is much higher. For 200×50 the maximal universe LP at $a_{\max} = 50$ is 637.7, so the heuristic's 786 is 148 rows above a real relaxation, not 744 rows above 42.

input	$a_{\max}$	m	best-cov	$\lvert C\rvert$	$\lvert C\rvert$ at $\Gamma = 64$
random 10^6	8	1 000 001	1 000 001	1 000 001	1 000 001
100 % 200×50	50	10 001	42	786	786
100 % 100×100	100	10 001	42	999	1741
100 % 50×200	200	10 001	42	958	1071
50 % 100×100	100	20 001	10 041	10 731	11 645
10 % 100×100	100	100 001	90 041	90 731	91 645
10 E. coli	8	52 117 680	26 094 932	30 298 453	30 298 453

Table 1: Automatic $a_{\max}$ plus the wrap-preserving cache cap. Motif length P becomes $a_{\max}$ on every tandem; the cap change moves only the files with $a_{\max} > 64$. Ten E. coli genomes share a 10.4 Mbp homologous spacing that fails the spike test, so auto is the default 8.

The mixed 10 %/50 % files of motif 50 and 200 do not move: their unique prefix already sits on best-cov, and the tandem leftover is packed by small adds (or by a wrap that $\Gamma = 64$ already held). The 100×100 mixed files pick up the same wrap as the pure tandem, -914 rows.

Two caveats follow from the same measurements. First, auto chooses the universe; it does not choose the packing. Phase I of greedy-degree on 200×50 still keeps 5633 ranks and relies on Phase II to reach 786. Second, enlarging LNS clusters (`GCSA_LNS_AUTO=1`) closes more of the 786–638 gap on that file ($|C| = 672$) and *opens* it on 100×100 wrap ($999 \to 1272$). The wrap-preserving cap is the change that is safe to leave on; cluster sizing remains instance-dependent.

5 Preference selection and the pseudoforest

5.1 Preferences

For every name c with $\mathrm{isize}(c) > 2$, `tree-dp` computes the full static candidate list $\hat{\mathcal{K}}_c$ and records a single **preferred** link

$$\pi(c) \;=\; \arg\max_{k \in \hat{\mathcal{K}}_c} \big(\mathrm{cov}(k),\; a_k\big)$$

in lexicographic order: maximum coverage, ties broken towards the *larger* shift (`pick_best_`). If the best candidate has coverage below κ — or if the list is empty — the name has no preference. Note the threshold $\mathrm{isize}(c) > 2$ rather than > 1: an interval of size 2 cannot host a link of coverage 3 anyway.

The tie-break towards larger a looks cosmetic. It is not: it decides which *name* a preference points at, and hence the shape of the graph built next. We will see it change the answer in §11.3.

5.2 The preference graph

Draw a directed edge $c \to w$ whenever the source run of $\pi(c)$ lies inside I_w. The following is the structural fact on which the entire algorithm rests.

Proposition 1 (Out-degree at most one)**.** *Each name has at most one outgoing preference edge, and the target w is well defined.*

Proof. A name has at most one preferred link, so at most one edge. For well-definedness we must show the source run $S = [s_{\mathrm{lo}}, s_{\mathrm{hi}})$ of a candidate lies inside a single interval. By construction the run was cut wherever $\mathrm{lcp}[u] < a + 1$, so all ranks of S agree in their first $a + 1$ gapped symbols. Since $a \geq 1$, they agree in at least their first 2 symbols, hence certainly in their first symbol, which is precisely the name. As intervals are the maximal blocks of constant first symbol, $S \subseteq I_w$ for exactly one w. □

Corollary 2 (Pseudoforest)**.** *The preference graph, viewed as an undirected graph, is a **pseudoforest**: every connected component contains at most one cycle.*

Proof. A component with ν vertices has at most ν edges, since each vertex contributes at most one out-edge. A connected graph on ν vertices with at most ν edges has cyclomatic number at most 1. □

This is the whole reason a dynamic program is available. An arbitrary "c can source from w" relation is a dense digraph on $\mathcal{N}$ and optimizing over it is the set-packing problem of `tree_dp3` §1.4, which is NP-hard in general. Restricting each name to one preferred link collapses that digraph to a functional graph, and functional graphs are one edge-deletion per component away from forests, where dynamic programming is routine.

Two degenerate cases deserve names.

- **Self-loops.** If $\pi(c)$ is an intra-interval link (§3.3) then $w = c$ and the edge is a self-loop.
- **Dangling sources.** If I_w is a singleton it is not in $\mathcal{N}$ and the edge points at a vertex the algorithm does not model; the edge is dropped (`by_name.count(s)`).

Algorithm 2 Forest construction from preferences

Input: preferences $\pi(\cdot)$, intervals by name
Output: a parent map inducing a forest

parent $\leftarrow \emptyset$
for all names c with a preference, **in ascending name code order do**
    $w \leftarrow$ the name whose interval contains the source of $\pi(c)$
    if $\mathrm{cov}(\pi(c)) < \kappa$ **then continue**
    end if
    if $w = c$ **then continue** ▷ intra-interval link: self-loop
    end if
    if $\mathrm{isize}(w) \leq 1$ **then continue** ▷ source in a singleton interval
    end if
    if HasAncestor(w, c) **then continue** ▷ would close a cycle: *discard*
    end if
    parent$[c] \leftarrow w$
end for

6 Cycle breaking

The DP needs a forest, so one edge per cycle must go. The implementation does this while installing edges, in a single pass:

`HasAncestor`(w, c) walks the parent chain up from w and reports whether it meets c. Since the partial structure is always a forest, the walk terminates and costs $O(\text{depth})$.

Two properties of this loop matter.

Fact 1 (The cut edge is chosen by name order, not by value). For a cycle $c_1 \to c_2 \to \cdots \to c_\nu \to c_1$, the edge discarded is the out-edge of whichever cycle vertex the loop visits last. The loop visits names in ascending arithmetic code order — the integer encoding of the k-mer, i.e. essentially alphabetical order of the gapped k-mer string. Nothing about the coverages, shifts or interval sizes of the cycle's edges enters the decision.

The comment in the source calls the iteration order "deterministic ... so parallel pref build does not change which cycle-breaking edges survive". That is true and worth having — the result is reproducible — but reproducibility is not optimality. A cycle of ν edges offers ν distinct forests and the algorithm takes the one indexed by the alphabet.

Fact 2 (This is where the loss is). `tree-dp` is exact on the model it constructs (Theorem 3). Optimality can therefore only be lost when the model is built. There are exactly three lossy steps:

1. each name is restricted to its *single* preferred link (§5);
2. each name is restricted to its *single* parent, so a name that could serve as source for several dependents' second-choice links cannot;
3. one edge per cycle is deleted by Fact 1.

Section 12 exhibits the third loss in the sharpest form available: twenty-four instances that are all the same optimization problem up to renaming, on which `tree-dp` returns two different answers depending only on the alphabet. Section 13 recalls a second, smaller instance where the same restriction bites and where — unlike in §12 — no later pass can repair it.

7 The KEEP/COMPRESS dynamic program

7.1 States and recurrence

Each forest node v is a name with interval size $\text{isize}(v)$ and (if it has a preference) a candidate $\pi(v)$ of coverage $\text{cov}(\pi(v))$ whose source lies in $\text{parent}(v)$'s interval. The DP decides, for each v, whether to

KEEP store all $\text{isize}(v)$ ranks of I_v literally, or

COMPRESS accept $\pi(v)$, storing $\text{isize}(v) - \text{cov}(\pi(v))$ ranks.

The two decisions interact along forest edges in one direction only. If v compresses, its source ranks lie in $I_{\text{parent}(v)}$ and must survive, which by (F2) forbids *every child of* v from covering them. The implementation takes the conservative reading: a compressing node forbids its children from compressing at all. This is what the second state records.

Definition 2 (DP states)**.** $\text{DP}(v, \mathit{src_ok})$ is the minimum number of stored ranks in the subtree of v, where $\mathit{src_ok} = \mathsf{true}$ means v is still allowed to accept $\pi(v)$.

$$\text{DP}(v, \mathsf{true}) = \min\Big(\underbrace{\text{isize}(v) + \sum_{w \in \text{ch}(v)} \text{DP}(w, \mathsf{true})}_{\text{KEEP}},\ \underbrace{\text{isize}(v) - \text{cov}(\pi(v)) + \sum_{w \in \text{ch}(v)} \text{DP}(w, \mathsf{false})}_{\text{COMPRESS}}\Big), \quad (4)$$

$$\text{DP}(v, \mathsf{false}) = \text{isize}(v) + \sum_{w \in \text{ch}(v)} \text{DP}(w, \mathsf{true}). \quad (5)$$

The COMPRESS branch is available only when $\mathit{src_ok}$ holds and $\text{cov}(\pi(v)) \geq \kappa$. Ties go to KEEP (the code takes COMPRESS only on *strict* improvement), which is the right default: keeping a hub free leaves options open for the leftover pass and Phase II.

7.2 The root rule

A root has no parent inside the forest, so the source of its preferred link lies elsewhere. It may still compress, but only if that "elsewhere" is not inside its own tree — otherwise accepting the link would pin ranks that the subtree DP has already decided to remove. The code checks exactly this:

$$\text{root } \rho \text{ may COMPRESS} \iff \text{cov}(\pi(\rho)) \geq \kappa \ \wedge\ w \neq \rho \ \wedge\ \neg\texttt{HasAncestor}(w, \rho),$$

where w is the source's name. In practice the check almost always fails, for a structural reason: a root is a root precisely because its own preference edge was the one discarded in Algorithm 2, and the edge was discarded because it closed a cycle — meaning w *is* a descendant. Every root in every example of this note reports `comp=n/a`.

7.3 Intra-interval links are invisible to the DP

Combining §3.3 with Algorithm 2 gives a consequence worth stating on its own.

Fact 3 (The forest DP cannot use an intra-interval link). Let $\pi(c)$ be an intra-interval link. Then $w = c$, so Algorithm 2 skips the edge (self-loop) and c becomes a root; and the root rule rejects COMPRESS because it requires $w \neq \rho$. So c is forced to KEEP by the DP regardless of how good $\pi(c)$ is. Intra-interval links can enter the solution only through the leftover greedy (§8.2) or Phase II (§9).

Example 1 is the extreme case. Running $\texttt{A}^{16}$ with tracing on:

```
=== tree-dp forest (src -> dependents) ===
roots: AA
root AA: keep=14 comp=n/a
[tree-dp] accept chosen...
[tree-dp] leftover greedy...
after Phase II: kept=11 accepted=1
```

The DP accepts nothing — its only node is a root whose preference is a self-loop — and the entire compression, from 17 down to the optimal 11, is performed by the leftover greedy. On this instance `tree-dp` is the leftover greedy.

7.4 Exactness on the forest

Theorem 3 (The DP is exact for its model)**.** *Fix the forest F and the preference assignment π. Among all subsets X of $\{\pi(c) : c \in F\}$ such that no node of X has a parent in X — equivalently, all* Keep/Compress *labellings in which no compressing node has a compressing child — the DP returns one of minimum $|C|$, and its value equals the sum over roots of* $\mathrm{DP}(\rho, \cdot)$ *plus* isize *of the names outside F.*

Proof. Subtrees rooted at distinct children of a node are vertex-disjoint and the only coupling between a node's decision and its subtrees' is through *src_ok*, which is a single bit determined by the parent's decision. Hence the cost of a labelling decomposes as $\sum_v \big(\mathrm{isize}(v) - [\,v \text{ compresses}\,] \cdot \mathrm{cov}(\pi(v))\big)$ and the feasible labellings are exactly those closed under "no compressing child of a compressing node". Equations (4)–(5) are the standard weighted independent-set-in-a-tree recurrences for that constraint, evaluated bottom-up; induction on subtree height gives the claim. □

The theorem is deliberately narrow. It says nothing about links other than $\pi(c)$, nothing about the discarded cycle edges, and nothing about names with no preference. Those are exactly the gaps of Fact 2.

7.5 Memoization and parallelism

Each root's tree is solved independently. Within a tree, states are memoized in a dense array `memo[2*v + src_ok]` indexed by a compact node id, with cost < 0 marking "unevaluated"; the recursion is a `std::function` closure over that array. Each of the $2|F|$ states is computed once and each edge is relaxed a constant number of times, so a tree of ν nodes costs $O(\nu)$ time and $O(\nu)$ memory.

Because the trees are vertex-disjoint, the per-root solve is embarrassingly parallel and the implementation runs it under `gcsa_parallel_for` with `GCSA_THREADS` workers (default: hardware concurrency). Two consequences:

- *Determinism is preserved.* The forest itself is built serially in name order (Algorithm 2) precisely so that the parallel preference enumeration cannot perturb which cycle edges survive. Roots are sorted before the parallel loop, and each root writes only its own slot of `chosen_by_root`.
- *The trace goes quiet.* `GCSA_TRACE_DP=1` prints per-root Keep/Compress values only when `GCSA_THREADS=1`, since interleaved `fprintf` from worker threads would be unreadable. Every trace in this note was therefore taken single-threaded.

The same `gcsa_parallel_for` covers the preference enumeration, which is where nearly all the wall time actually goes (§15). The DP itself is never the bottleneck.

Algorithm 3 Materialize the DP's decisions

```
1: function AcceptSubtree(v)
2:     for all w ∈ ch(v) do AcceptSubtree(w)
3:     end for
4:     if v was not chosen by the DP then return
5:     end if
6:     c ← π(v)
7:     if ¬Available(c) then
8:         c ← first available candidate of v in (cov ↓, a ↓) order
9:     end if
10:    if cov(c) ≥ κ then Accept(c)
11:    end if
12: end function
```

8 Materialization: the accept pass and the leftover greedy

8.1 Accepting deepest-first

The DP produces a set of *chosen* names, but choosing is not accepting: accepting mutates the global `removed_` and `pin_count_` arrays, and the order in which the mutations happen can matter. The implementation walks each tree with `accept_subtree`, recursing into children *before* handling the node:

Deepest-first ordering means dependents place their pins on a hub before the hub is looked at. The DP has already guaranteed the two cannot collide — a compressing node's children were forced to Keep — so the fallback on line 5 is a belt-and-braces measure. It fires when the static preference has become unavailable for a reason the DP does not model, and it re-picks from the cached, coverage-sorted candidate list rather than re-enumerating.

`Accept` itself is three lines: increment `pin_count_` over the source run (recording an owner for the first pin), set `removed_` over the covered ranks and decrement `kept_count_`, and store the candidate in the `accepted` map under its name.

8.2 The leftover greedy

After the accept pass, many names may still have no link:

- names with isize ≤ 2, which never got a preference;
- names whose preference had coverage below κ;
- names whose preferred link is intra-interval (Fact 3);
- names the DP deliberately left at Keep;
- roots, which almost always cannot compress.

The last two categories are the large ones, and they are not leftovers in any pejorative sense — a hub the DP kept is a hub the DP decided was worth more as a source than as a target, and it may well still be compressible *in addition*, using a different source.

The leftover pass therefore sweeps all names without an accepted link in descending interval size and, for each, takes the first candidate from the cached list that is available right now — source fully kept, covered ranks neither removed nor pinned. The cache is already sorted by $(\text{cov}\downarrow, a\downarrow)$, so "first available" is "best available". No re-enumeration happens.

This pass is not a footnote. On $\texttt{A}^{16}$ it does 100 % of the work (§7.3); on the main worked example of §11 it contributes 3 of the 10 removed ranks and is the difference between the optimum and missing it.

9 Phase II: the unpin/retarget loop

Phase II is shared with `dep-order` (and, in `tree-dp3`, runs before the cluster solve). It is a local-improvement fixed point over the accepted set.

9.1 The retarget move

Consider accepting a better candidate c for name x. The obstacle is that some ranks of T_c may be pinned: another accepted link d is using them as *its* source. Greedily this is a dead end. But the pin can often be moved.

Suppose S_d is a contiguous sub-run of T_c, say S_d corresponds to positions $[t_0, t_0 + |S_d|)$ of T_c. Every rank of T_c is the a_c-shift of the corresponding rank of S_c. So the ranks d was reading can equally be read from the corresponding sub-run of S_c, provided d's shift absorbs c's:

$$S'_d \;=\; \big[\, S_c\text{'s } t_0\text{-th rank},\ \ldots \big), \qquad a'_d \;=\; a_d + a_c. \tag{6}$$

This is `can_retarget_` and the assignment `d.add += cand.add` in `try_accept_with_retarget_`. If every dependent pinning into T_c can be retargeted this way, the move is applied: revoke x's old link if any, revoke and re-accept each dependent with its new source and shift, then accept c.

Fact 4 (Phase II synthesizes links outside the candidate universe). The shift $a'_d = a_d + a_c$ of (6) can exceed $a_{\max}$. Such a link is never produced by `enumerate_candidates_` and never appears in any cached candidate list; it exists only as a consequence of a retarget.

This is easy to see in the output. On `r50-600.fasta` with $S =$ ##### at the default $a_{\max} = 8$, the final hash table contains 442 H_{offset} entries, of which 168 **carry a shift greater than** 8, the largest being 30 — nearly four times the enumeration ceiling. On `r300-300.fasta` it is 116 of 237. Composite adds are not a curiosity; on repetitive inputs they are a third to a half of the solution.

Fact 4 is also the reason `tree-dp3` keeps Phase II rather than replacing it with its exact cluster solve: the cluster solve is exact over the *enumerated* universe, which does not contain composite links.

9.2 The dirty-set work queue

Phase II is organized in *generations*. The intervals with isize > 1 are sorted once by descending size. The first generation's dirty set is all of them. Within a generation each interval is visited at most once and, for the current best cached candidates in coverage order, a retarget-accept is attempted; the first success wins.

A visit exits cheaply in two common cases: the name is already *saturated* (its coverage equals its interval size, so nothing can improve) or its best cached candidate does not beat its current coverage (`skip_best`).

The next generation's dirty set is not "everything again". It is seeded, and only if something improved during this generation, from

- intervals whose attempt failed because of a pin (`pin_blocked`) — pins may have moved since; and
- intervals still short of their cached best (`still_open`) — either they accepted something suboptimal or their preferred source was not kept at the time.

Names that were saturated or genuinely exhausted are never re-queued. The loop ends when the next dirty set is empty (a fixed point) or the budget runs out.

9.3 The budget

Phase II runs at most `max_iters` generations, resolved in the precedence order

$$\texttt{-phase2-iters} > \texttt{GCSA_PHASE2_MAX_ITERS} > \texttt{kPhase2DefaultIters} = 100,$$

with the CLI flag winning if positive. The timing line reports which source was used and why the loop stopped (`fixed-point`, `max-iters`, or `adaptive-stall`). Two environment knobs, `GCSA_PHASE2_STALL` and `GCSA_PHASE2_MIN_GAIN`, enable an adaptive early stop that cuts long plateaus in which coverage rises but $|C|$ does not; it is off by default.

In practice the budget is enormous relative to what is needed. Measured at `7f9127b` with $S = \#\#\#\#\#$:

input	generations	stop	tries	improves
`r6-12.fasta`	1	fixed-point	15	0
`r600-60.fasta`	2	fixed-point	18	10
`r50-600.fasta`	2	fixed-point	159	130
`r300-300.fasta`	2	fixed-point	94	86

The pattern is invariable: generation 1 does the work, generation 2 confirms there is nothing left, and the loop exits at a fixed point far under the budget of 100. The 9 Mbp `r300-3000.fasta` under the gapped shape `####.####` also stops at a fixed point after two generations. Lowering the budget below the fixed point is the only way to make it bind, and doing so costs $|C|$.

9.4 Why Phase II is invisible on small instances

Phase II never fires in any of the worked examples below, and this is not cherry-picking. Scanning 700 randomly generated repetitive texts over alphabets $\{A, C\}$, $\{A, C, G\}$ and $\{A, C, G, T\}$ with motif lengths 4–8, lengths 18–30 and the five shapes `#.#`, `##`, `#..#`, `##.#`, `##.##`, *no* instance with $m \leq 32$ produced a single `IMPROVE` or `REPLACE` in Phase II. The reason is structural: on an instance with a handful of names, the forest DP plus the leftover greedy already reach a state where every name is either saturated or has no better cached candidate, which is precisely Phase II's fixed-point condition. Phase II earns its keep when there are thousands of intervals and the accept order has stranded many of them, which is exactly when a hand-drawn walkthrough is impossible.

The honest way to present Phase II at small scale is therefore not a toy trace but the composite-add census of Fact 4 and the suite delta of §16: `tree-dp` minus Phase II is `tree-dp2`, and it costs 3800 positions suite-wide.

10 The algorithm in full

Algorithm 4 `tree-dp`

Input: gapped suffix array G, intervals $\{I_c\}$, $a_{\max}$, $\kappa = 3$
Output: compressed array C and hash table

Phase 0 — preferences (parallel over intervals)
for all c with isize$(c) > 1$ **do**
  $\hat{\mathcal{K}}_c \leftarrow$ EnumerateCandidates(c, `avail`=false)
  if isize$(c) > 2$ and $\max_k \mathrm{cov}(k) \geq \kappa$ **then**
    $\pi(c) \leftarrow$ PickBest($\hat{\mathcal{K}}_c$) ▷ max cov, ties to larger a
  end if
  sort $\hat{\mathcal{K}}_c$ by (cov $\downarrow, a \downarrow$) and cache
end for
Phase 1 — forest (serial, name order)
build parent by Algorithm 2; roots $\leftarrow$ nodes without a parent
Phase 1 — DP (parallel over roots)
for all roots ρ **do**
  evaluate (4)–(5) with memoization
  root may Compress only if its preferred source is outside its tree
  record the chosen names of this tree
end for
Phase 1 — materialize
for all roots ρ **do** AcceptSubtree(ρ) ▷ Algorithm 3
end for
for all c with isize$(c) > 1$ and no accepted link, by isize $\downarrow$ **do**
  accept the first available candidate of $\hat{\mathcal{K}}_c$, if any
end for
Phase II — unpin/retarget
run the dirty-generation fixed point of §9
Finalize
$C \leftarrow$ kept ranks in rank order; fill $H_{\text{offset}}/H_{\text{rest}}$ per name

11 Worked example: `(AGCT)`^{5} with shape `#.#`

11.1 Why this instance

The natural choice would be the family's running example, $T = (\texttt{GCCTTTAAAG})^3$ with $S = \#.\#$. It is a poor vehicle for `tree-dp`, for a specific reason: its preference graph is two disjoint stars with no cycle, its four leaves compress and its two hubs keep, its leftover pass finds nothing and its Phase II is a no-op. Four of the seven stages of Algorithm 4 do nothing. It is covered for continuity in §14.

$T = (\texttt{AGCT})^5$ with the same shape is barely larger ($m = 21$ against 31, and 7 names against 9) and exercises far more:

- every compressible name has several candidates, so preference selection is a real choice — and one of them is settled by the *tie-break*, not by coverage;
- the preference graph has a genuine 3-cycle plus a pendant, so cycle breaking fires;
- the DP performs a non-trivial Keep versus Compress comparison at an internal node (6 against 9) and propagates a forced Keep;

- the leftover greedy contributes 3 of the 10 removed ranks, and one name is blocked in that pass by pins its own dependents placed;
- the result, $|C| = 11$, is the ILP optimum, so the walkthrough ends somewhere defensible;
- and it has an isomorphic twin (§12) on which the same algorithm returns 13.

Only Phase II is idle, for the reason given in §9.4.

11.2 The instance

$$T = \texttt{AGCTAGCTAGCTAGCTAGCT}, \quad n = 20, \quad S = \#.\#, \quad s = 3,\ w = 2, \quad m = 21.$$

Position p carries the name $T[p]T[p{+}2]$. There are 7 names, of which 4 are compressible.

Table 2: The gapped suffix array of $(\texttt{AGCT})^5$ under `#.#`, from `./validate`. Singleton intervals are `$$`, `C$`, `T$`.

rank	orig	name	lcp	rank	orig	name	lcp
0	20	`$$`	0	11	17	`GT`	0
1	16	`AC`	0	12	13	`GT`	1
2	12	`AC`	1	13	9	`GT`	2
3	8	`AC`	2	14	5	`GT`	3
4	4	`AC`	4	15	1	`GT`	5
5	0	`AC`	5	16	19	`T$`	0
6	18	`C$`	0	17	15	`TG`	0
7	14	`CA`	0	18	11	`TG`	1
8	10	`CA`	2	19	7	`TG`	3
9	6	`CA`	3	20	3	`TG`	4
10	2	`CA`	4				

So

$$I_{\texttt{AC}} = [1, 6), \quad I_{\texttt{CA}} = [7, 11), \quad I_{\texttt{GT}} = [11, 16), \quad I_{\texttt{TG}} = [17, 21),$$

with sizes $5, 4, 5, 4$ and three singletons, totalling 21. If nothing is compressed, $|C| = 21$.

11.3 Phase 0: the candidates

At $a_{\max} = 8$, `enumerate_candidates_` produces nine candidates in total. (These were extracted from the LP the ILP baseline writes when invoked with `-universe legacy`, which is by construction exactly the heuristics' view.)

Read one row to see the machinery. The starred `TG` candidate has source $[2, 6)$ — ranks $2, 3, 4, 5$ of $I_{\texttt{AC}}$, i.e. text positions $12, 8, 4, 0$ — shift $a = 1$, hence covered text positions $15, 11, 7, 3$, which are ranks $17, 18, 19, 20$: all of $I_{\texttt{TG}}$. The source run survives the lcp $\geq a + 1 = 2$ cut because lcp[3] $= 2$, lcp[4] $= 4$, lcp[5] $= 5$; it stops at rank 2 going left because lcp[2] $= 1 < 2$.

Three of the four preferences are decided by coverage alone. The fourth is not:

> **`CA` is decided by the tie-break.** Both of its candidates have coverage 3, and `pick_best_` prefers the larger shift, so $\pi(\texttt{CA})$ is the $a = 2$ candidate, whose source lies in $I_{\texttt{AC}}$ — *not* the $a = 1$ candidate, whose source lies in $I_{\texttt{TG}}$.

Had the tie gone the other way, the preference edge would have been `CA` $\to$ `TG` instead of `CA` $\to$ `AC`, and the graph of Figure 1 would have had no cycle at all.

Table 3: The candidate universe `tree-dp` sees on $(\texttt{AGCT})^5$. The preferred candidate of each name is starred.

	target	cov	a	source run	source name	covered ranks
⋆	AC	4	1	$[12, 16)$	GT	$1, 2, 3, 4$
	AC	3	2	$[8, 11)$	CA	$1, 2, 3$
	CA	3	1	$[18, 21)$	TG	$7, 8, 9$
⋆	CA	3	2	$[3, 6)$	AC	$7, 8, 9$
⋆	GT	4	1	$[7, 11)$	CA	$11, 12, 13, 14$
	GT	3	2	$[18, 21)$	TG	$11, 12, 13$
	GT	3	3	$[3, 6)$	AC	$11, 12, 13$
⋆	TG	4	1	$[2, 6)$	AC	$17, 18, 19, 20$
	TG	3	2	$[13, 16)$	GT	$17, 18, 19$

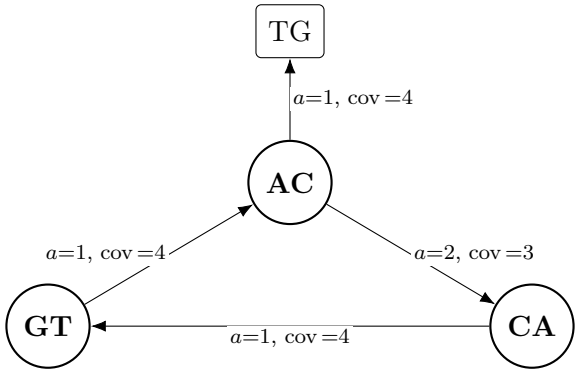


Figure 1: The preference graph of $(\texttt{AGCT})^5$. An edge $w \to c$ means "c's preferred link sources from I_w". Four vertices, four edges, out-degree one everywhere: a pseudoforest with exactly one cycle, AC → CA → GT → AC, and one pendant.

11.4 Phase 1: the preference graph

11.5 Phase 1: cycle breaking

Algorithm 2 visits names in ascending code order. The codes here are AC = 3, CA = 7, GT = 15, TG = 19, so the order is AC, CA, GT, TG.

step	name	would set	outcome
1	AC	parent[AC] = GT	installed (GT has no ancestors yet)
2	CA	parent[CA] = AC	installed
3	GT	parent[GT] = CA	**discarded**: CA → AC → GT
4	TG	parent[TG] = AC	installed

At step 3, `HasAncestor(CA, GT)` walks CA → AC → GT and finds GT, so the edge would close the cycle and is dropped. GT keeps its preferred candidate as a *value* — it just has no parent, so the DP will not be able to use it. The trace confirms the forest:

```
=== tree-dp forest (src -> dependents) ===
  AC -> CA TG
  GT -> AC
roots: GT
```

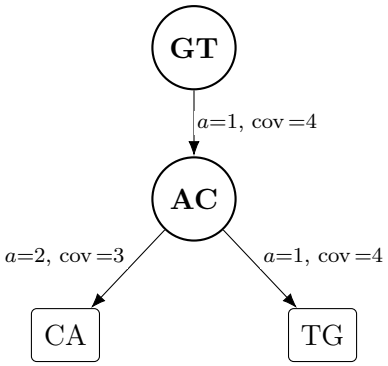


Figure 2: The forest after cycle breaking. One tree, root `GT`.

11.6 Phase 1: the DP

The DP runs bottom-up on Figure 2. Interval sizes are isize(`GT`) = 5, isize(`AC`) = 5, isize(`CA`) = 4, isize(`TG`) = 4.

Table 4: The DP table for (`AGCT`)^5. Entries are (KEEP, COMPRESS) costs with the winner in bold; "—" marks an unavailable COMPRESS branch.

node	state	KEEP	COMPRESS	decision
`CA` (leaf)	*src_ok* = true	4	$\mathbf{4-3=1}$	COMPRESS
`CA` (leaf)	*src_ok* = false	**4**	—	KEEP
`TG` (leaf)	*src_ok* = true	4	$\mathbf{4-4=0}$	COMPRESS
`TG` (leaf)	*src_ok* = false	**4**	—	KEEP
`AC`	*src_ok* = true	$\mathbf{5+1+0=6}$	$5-4+4+4=9$	KEEP
`GT` (root)	—	$\mathbf{5+6=11}$	n/a	KEEP

The middle row is the interesting one and is exactly the comparison the whole construction exists to make. If `AC` compresses it saves its own 4 ranks, but it must then keep its source ranks alive, which forbids both children from compressing and costs $1+0 \to 4+4$, a net loss of 3. So `AC` keeps. The root `GT` reports `comp=n/a` because its preferred source, `CA`, is a descendant — the root rule of §7. The trace agrees:

```
root GT: keep=11 comp=n/a
```

The DP's value 11 is the cost of the *tree*, whose four names hold $5+5+4+4=18$ ranks; it saves $18-11=7$. Adding the three singleton ranks gives 14 stored positions after the accept pass. (That this eventually coincides with the final $|C|=11$ is a coincidence of this instance.)

11.7 Phase 1: accept and leftover

`AcceptSubtree` recurses deepest-first from the root, so the order is `CA`, `TG`, `AC`, `GT`; only the first two were chosen:

```
[tree-dp] accept chosen...
  ACCEPT CA <- AC add=2 cov=3
  ACCEPT TG <- AC add=1 cov=4
```

State after the accept pass:

- removed: ranks $7, 8, 9$ (of `CA`) and $17, 18, 19, 20$ (of `TG`); $kept = 21 - 7 = 14$;
- pinned: `CA`'s link sources $[3, 6)$ and `TG`'s sources $[2, 6)$, so ranks $2, 3, 4, 5$ of $I_{\texttt{AC}}$ carry pins.

The leftover pass now considers the two names with no link, `AC` and `GT` (both of size 5).

`AC` gets nothing. Its two candidates cover $\{1, 2, 3, 4\}$ and $\{1, 2, 3\}$ respectively. Ranks $2, 3, 4$ are pinned, because `AC`'s own dependents are reading them. Neither candidate is available. *The hub is protected from itself*, and this happens automatically through the pin counter — nothing in the DP or the accept pass had to arrange it.

`GT` gets its third-choice link. Walking the cached list in $(\text{cov}\downarrow, a\downarrow)$ order:

- $\text{cov} = 4$, $a = 1$, source $[7, 11) \subseteq I_{\texttt{CA}}$: ranks $7, 8, 9$ were removed, so the source is not kept. Rejected by (F2).
- $\text{cov} = 3$, $a = 3$, source $[3, 6) \subseteq I_{\texttt{AC}}$: source fully kept; covered ranks $11, 12, 13$ neither removed nor pinned. **Accepted.**

That last acceptance removes three more ranks, bringing *kept* to 11. Note that all three accepted links now source from $I_{\texttt{AC}}$ — which is legal precisely because (F2) constrains sources against *covered* ranks, not against each other (`tree_dp3`, Lemma 5(b)).

11.8 Phase II and the result

Phase II visits the four compressible names, finds every one either saturated or without a better available candidate, and exits at a fixed point in one generation having changed nothing.

The final hash table, printed by `-table`:

name	code	$H_{\text{offset}}(pos, a, num)$	$H_{\text{rest}}(pos, num)$
`$$`	0	nil	$(0, 1)$
`AC`	3	nil	$(1, 5)$
`C$`	6	nil	$(6, 1)$
`CA`	7	$(3, 2, 3)$	$(7, 1)$
`GT`	15	$(3, 3, 3)$	$(8, 2)$
`T$`	16	nil	$(10, 1)$
`TG`	19	$(2, 1, 4)$	nil

The kept ranks, in rank order, are $0;\ 1, 2, 3, 4, 5;\ 6;\ 10;\ 14, 15;\ 16$, so

$$C = [\,20,\ 16,\ 12,\ 8,\ 4,\ 0,\ 18,\ 2,\ 5,\ 1,\ 19\,], \qquad |C| = 11.$$

Every decode can be checked by hand against this array. For `TG`, $H_{\text{offset}} = (2, 1, 4)$ reads $C[2..5] = 12, 8, 4, 0$ and adds $1 \cdot s = 3$, giving $15, 11, 7, 3$: the four `TG` positions. For `CA`, $(3, 2, 3)$ reads $C[3..5] = 8, 4, 0$ and adds 6, giving $14, 10, 6$; the fourth `CA` position, 2, is stored literally at $C[7]$, which is what $H_{\text{rest}} = (7, 1)$ says. For `GT`, $(3, 3, 3)$ reads the same three source positions and adds 9, giving $17, 13, 9$, with 5 and 1 kept at $C[8], C[9]$.

Finally, from `ilp_baseline`:

$$\text{optimal } |C| = 11 \quad \text{versus} \quad \texttt{greedy} = 14,\ \texttt{dep-order} = 11,\ \texttt{tree-dp} = 11.$$

`tree-dp` is optimal here, and the leftover pass was indispensable: without it the answer would have been 14.

12 The isomorphic twin: where cycle breaking costs two positions

Now take $T = (\texttt{ACGT})^5$ — the same four letters, permuted — with the same shape. Again $m = 21$, again seven names, again four compressible ones of sizes $5, 5, 4, 4$.

12.1 The two instances are the same problem

Consider the relabelling

$$\texttt{AC} \mapsto \texttt{AG}, \quad \texttt{CA} \mapsto \texttt{GA}, \quad \texttt{GT} \mapsto \texttt{CT}, \quad \texttt{TG} \mapsto \texttt{TC}.$$

Under it, the interval sizes correspond, and — checked mechanically over the *full* link universe of both instances, all 15 candidates each, comparing shifts, coverages, source names, and source/covered positions relative to their intervals — the two candidate sets correspond exactly. As optimization problems in the sense of §2, $(\texttt{AGCT})^5$ and $(\texttt{ACGT})^5$ under `#.#` are the same instance with the names renamed. In particular both have optimum $|C| = 11$.

The same check run over all 24 permutations σ of the alphabet, comparing $(\sigma_1\sigma_2\sigma_3\sigma_4)^5$ against $(\texttt{ACGT})^5$ under the induced relabelling, reports that *every one* of the 24 is isomorphic to the reference instance, and `ilp_baseline` gives optimum 11 for all 24. Yet:

instance	optimum	`greedy`	`dep-order`	`tree-dp`	`tree-dp3`
$(\texttt{AGCT})^5$	11	14	11	**11**	11
$(\texttt{ACGT})^5$	11	14	11	**13**	11

and across the whole orbit:

`tree-dp`	`tree-dp2`	`tree-dp3`	`dep-order`	`greedy`	permutations
11	11	11	11	14	`AGCT ATCG ATGC CGAT CTAG CTGA GTAC GTCA`
13	13	11	11	14	the other sixteen

`greedy`, `dep-order` and `tree-dp3` each return the same value on all 24 isomorphic instances, as they must if their answer depends only on the problem. `tree-dp` and its Phase-II-less variant `tree-dp2` split the orbit one third / two thirds according to how the four gapped k-mers happen to sort.

12.2 What differs

Only the arithmetic codes, and hence the visit order of Algorithm 2:

$$\underbrace{\texttt{AC}\,(3) < \texttt{CA}\,(7) < \texttt{GT}\,(15) < \texttt{TG}\,(19)}_{(\texttt{AGCT})^5} \qquad \text{versus} \qquad \underbrace{\texttt{AG}\,(4) < \texttt{CT}\,(10) < \texttt{GA}\,(12) < \texttt{TC}\,(18)}_{(\texttt{ACGT})^5}.$$

Translating the second through the relabelling gives the order $\texttt{AC}, \texttt{GT}, \texttt{CA}, \texttt{TG}$ — the middle two swapped. That is the entire difference between the two runs.

The consequence is a different edge cut and a different forest shape:

12.3 Why the path loses

The DP on Figure 3:

$$\begin{aligned}
\mathrm{DP}(\texttt{TC}, \mathsf{true}) &= \min(4,\ 4-4) = 0, & \mathrm{DP}(\texttt{TC}, \mathsf{false}) &= 4,\\
\mathrm{DP}(\texttt{AG}, \mathsf{true}) &= \min(5+0,\ 5-4+4) = 5 \ \ (\text{tie} \Rightarrow \textsc{Keep}), & \mathrm{DP}(\texttt{AG}, \mathsf{false}) &= 5+0 = 5,\\
\mathrm{DP}(\texttt{CT}, \mathsf{true}) &= \min(5+5,\ 5-4+5) = 6 \ \ (\textsc{Compress}),\\
\mathrm{DP}(\texttt{GA}) &= 4+6 = 10 \ \ (\textsc{Keep},\ \texttt{comp=n/a}).
\end{aligned}$$

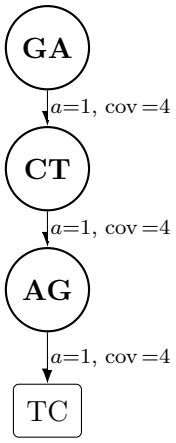


Figure 3: The forest `tree-dp` builds on $(\texttt{ACGT})^5$: a path, not a star. The discarded edge is parent[GA] = AG (coverage 3), because GA is visited third and by then AG → CT → GA already exists.

The trace prints `root GA: keep=10 comp=n/a`, and the accept pass takes TC and CT, coverage $4 + 4 = 8$, leaving *kept* = 13.

Now the leftover pass runs and, unlike in §11, finds nothing:

- AG's candidates cover ranks $\{1, 2, 3, 4\}$ and $\{1, 2, 3\}$; TC's accepted link sources $[2, 6)$, pinning $2, 3, 4, 5$. Blocked.
- GA's two candidates both cover $\{12, 13, 14\}$; but CT *compressed*, and its link sources $[12, 16)$, pinning all of $12, 13, 14, 15$. Blocked.

Phase II then confirms a fixed point. Final $|C| = 13$.

The single decision that separates the two runs is at DP(CT): on the path, compressing CT is worth 6 against 10, so the DP takes it — and in doing so pins $I_{\texttt{GA}}$ solid, which is what starves the leftover pass. In the star of Figure 2 the corresponding node (AC) is an internal node with *two* children, so compressing it would cost two forced Keeps rather than one; it keeps, and its interval stays available as a source for the leftover pass to exploit.

12.4 Two restrictions, interacting

It would be too simple to blame the cut edge alone. `tree_dp3` §2.3 analyses this instance and shows that the optimal solution asks CT to take a link it does *not* prefer — coverage 3 at $a = 3$ sourcing from $I_{\texttt{AG}}$, rather than its preferred coverage-4 link at $a = 1$ sourcing from $I_{\texttt{GA}}$ — because moving its source off $I_{\texttt{GA}}$ is what frees GA to compress in turn. So restriction (1) of Fact 2 binds: no choice of cut edge lets the *DP* express the optimum, because the DP only ever offers a name its preferred link.

What saves the successful eight permutations is the pass the DP does not control. Translating the $(\texttt{AGCT})^5$ run of §11 through the relabelling, its three accepted links are

$$\texttt{TC} \leftarrow \texttt{AG}\ (a{=}1,\ \text{cov}\,4), \quad \texttt{GA} \leftarrow \texttt{AG}\ (a{=}2,\ \text{cov}\,3), \quad \texttt{CT} \leftarrow \texttt{AG}\ (a{=}3,\ \text{cov}\,3),$$

which is exactly the optimal triple of `tree_dp3` §2.3 — and the third of them is precisely the non-preferred link. The DP supplied the first two; the *leftover greedy* supplied the third, because the leftover pass walks the whole cached candidate list and is under no obligation to take the preferred entry. So the two restrictions compose as follows.

Remark 3. On this instance the DP alone can never reach 11, whatever the cut (restriction 1). The leftover pass *can* contribute the missing non-preferred link — but only if the cut left the

relevant interval unpinned, which is a property of the forest shape (restriction 3). Under a star, AG's dependents pin AG but leave CT free, and the leftover pass closes the gap; under a path, the DP's decision to compress CT pins all of $I_{\texttt{GA}}$ and the leftover pass finds nothing. Neither restriction is sufficient on its own to explain the 13.

tree-dp3 removes both at once, by re-solving the dependency cluster {AG, CT, GA, TC} exactly over the full enumerated universe after Phase II; it returns 11 on all 24 permutations. See tree_dp3 §2.3 and §5.

13 The smallest counterexample, and what no later pass can fix

The second worked instance is $T = (\texttt{AC})^6 = \texttt{ACACACACACAC}$ with $S = \#.\#$, $m = 13$. It is analysed in full in tree_dp3 §2.2 (Counterexample I); we recall it here because it isolates the cut-edge restriction more cleanly than §12 does, and because it is the case tree-dp cannot recover from.

Positions alternate: even positions carry AA, odd positions carry CC, giving $I_{\texttt{AA}} = [2, 7)$ and $I_{\texttt{CC}} = [8, 13)$, both of size 5, plus three singletons. A link shifts text positions by $3a$, and AA sits on even while CC sits on odd positions, so a must be odd and $a \geq 3$ overshoots: every link has $a = 1$. The enumerated universe has exactly two members,

$$k_1 : \texttt{AA} \leftarrow \texttt{CC},\ \text{cov} = 3, \qquad k_2 : \texttt{CC} \leftarrow \texttt{AA},\ \text{cov} = 4.$$

Each is the other's target, so the preference graph is a 2-cycle, and the two orientations are *not* worth the same: k_2 covers four ranks, k_1 only three. The sources of k_1 and k_2 each meet the other's covered set, so by (F2) at most one may be chosen and the optimum is $|C| = 13 - 4 = 9$, using k_2.

Algorithm 2 visits AA (code 2) before CC (code 8). It installs parent[AA] = CC and then discards the reverse edge, which would close the cycle. The forest is the single edge CC → AA with root CC, and

$$\text{DP}(\texttt{AA}, \texttt{true}) = \min(5,\ 5 - 3) = 2 \ \ (\textsc{Compress}), \qquad \text{DP}(\texttt{CC}) = 5 + 2 = 7 \ \ (\textsc{Keep},\ \texttt{comp=n/a}),$$

matching the traced `root CC: keep=7 comp=n/a`. The accept pass takes k_1, and $|C| = 13 - 3 = 10$.

The point is what the cut destroyed. Had the loop installed parent[CC] = AA instead — which it would have, on any instance whose two names sorted the other way — the tree would be AA → CC, the DP would compute $\text{DP}(\texttt{CC}, \texttt{true}) = \min(5, 5 - 4) = 1$ and $\text{DP}(\texttt{AA}) = 5 + 1 = 6$, accept k_2, and land on the optimum 9. The better of the two orientations was available, it was better by a whole covered rank, and the algorithm discarded it because AA < CC alphabetically.

Unlike §12, nothing downstream repairs this. After k_1 is accepted, AA is saturated and CC's only candidate k_2 has its source inside the ranks k_1 removed, so the leftover pass and Phase II both see a fixed point immediately. (dep-order does reach 9 here, because its accept order leaves CC the first move and its Phase II retarget finds the coverage-4 link; and greedy gets 10. No method in the family dominates another.)

The two counterexamples compare as follows.

	$(\texttt{AC})^6$	$(\texttt{ACGT})^5$
tree-dp / optimum	10/9	13/11
cut edge is decisive (3)	yes	yes
DP can express the optimum	under the other cut	under no cut
optimum needs a non-preferred link (1)	no	yes
a later pass rescues it	no	under the right cut

Restriction (2) of Fact 2 — one parent per name — is not isolated by either instance; it is the hardest of the three to exhibit at this scale, because it requires a name whose *second* choice of source would have paid.

14 Continuity: the walkthrough's running example

For readers coming from `dep_order_walkthrough`, here is $T = (\texttt{GCCTTTAAAG})^3$, $S = \#.\#$, $m = 31$, at `7f9127b`.

Nine names, six compressible: `AA`(3), `AG`(5), `CT`(6), `GC`(5), `TA`(6), `TT`(3). Preferences:

target	cov	a	source run	source name
`AA`	3	2	$[19, 22)$	`GC`
`AG`	5	2	$[11, 16)$	`CT`
`TA`	6	1	$[10, 16)$	`CT`
`TT`	3	1	$[19, 22)$	`GC`

`CT` and `GC` appear only as sources: they have *no candidates at all*. The walkthrough describes them as preferring each other with coverage 2; since `kMinCoverage` was raised to 3, `emit_run_` discards those links during enumeration and they no longer exist anywhere in the pipeline.

The rest is straightforward. The preference graph is two disjoint stars with no cycle, so nothing is cut. The forest is $\texttt{CT} \to \{\texttt{AG}, \texttt{TA}\}$ and $\texttt{GC} \to \{\texttt{AA}, \texttt{TT}\}$ with roots `CT` and `GC`; both roots report `comp=n/a` for the simplest possible reason, namely that they have no preference to compress with. All four leaves compress, the DP reports keep = 6 and keep = 5 for the two roots, the leftover pass finds nothing (the only names without links are `CT` and `GC`, which have no candidates), and Phase II is a no-op:

$$|C| = 31 - (5 + 6 + 3 + 3) = 14,$$

which the ILP baseline confirms is optimal. The instance is a clean illustration of hub/leaf structure and a poor illustration of the algorithm, which is why §11 uses a different one.

Remark 4 (Recommendation, not an edit). §6 of `dep_order_walkthrough` should be reduced to a short pointer at this document. Two of its statements are now stale — the coverage-2 preferences for `CT` and `GC`, and the implied "source outside I_c" rule that §3.3 replaced — and its Tree-DP figure shows edges that the current enumerator does not produce. What is worth keeping there is the construction of the gapped suffix array and the `greedy`/`dep-order` comparison, which this note does not duplicate. *This edit has not been made.*

15 Complexity and practical behaviour

15.1 Asymptotics

Let m be the number of ranks, $|\mathcal{N}|$ the number of compressible names and $a_{\max}$ the shift ceiling.

- **Preference enumeration.** For one interval and one shift, the scan is $O(\text{isize}(c))$ lookups plus an $O(\text{isize}(c) \log \text{isize}(c))$ sort; the intra-interval splitting is linear in the run length. Summed over all intervals and shifts, this is $O(a_{\max} \cdot m \log m)$ worst case and $O(a_{\max}\, m)$ in practice, since the sort is over short lists. It is done once and cached.
- **Forest construction.** $O(|\mathcal{N}| \cdot d)$, where d bounds the parent-chain depth walked by `HasAncestor`. Observed depths are single digits.
- **The DP.** $O(|F|)$ time and memory over forest nodes: each of the $2|F|$ states is evaluated once.
- **Accept and leftover.** $O(\sum_c \text{isize}(c)) = O(m)$ plus a scan of cached candidate lists.
- **Phase II.** $O(\text{generations} \times \text{tries} \times \text{cost of a retarget})$, with a retarget costing $O(\text{covered} + \text{dependents})$. Unbounded in principle; bounded in practice by the fixed point after two generations (§9).

15.2 Measured phase timings

On `r300-3000.fasta` ($n = 9{,}000{,}000$), with `GCSA_TIMING=1`:

Table 5: `tree-dp` phase timings in milliseconds, `r300-3000.fasta`, $a_{\max} = 8$.

shape	threads	pref	forest	dp	accept	leftover	Phase II
`#####`	1	1980	0.3	0.4	1.6	1.0	0.8
`#####`	8	579	0.4	0.5	1.8	1.0	0.5
`####.####`	1	838	15.7	15.7	29.9	26.4	32243
`####.####`	8	238	20.2	14.5	34.5	25.9	36730

Three things to read off.

Preference enumeration dominates when Phase II has nothing to do. On the contiguous shape it is 99.7 % of the compression time. This is the regime the parallelization was built for.

Phase II dominates otherwise. On the gapped shape it is over 97 % of the total, in absolute terms two orders of magnitude more than everything else combined — and it is serial. The gapped shape produces far more intervals with improvable links, and each retarget walks its dependents. Whole-run speedup from threading is therefore 3.4× on `#####` but nil on `####.####` — the two totals there differ by less than Phase II's own run-to-run variance — even though the parallel section speeds up by the same factor in both shapes. If `tree-dp`'s wall time is ever a problem, Phase II is the thing to attack.

The DP is free. Forest construction, the DP, the accept pass and the leftover pass together account for 88 ms of the gapped-shape run on a 9 Mbp input, and 3.3 ms of the contiguous one: 0.3 % and 0.2 % of their totals respectively. Whatever `tree-dp` costs, it is not the dynamic program.

15.3 Parallel speedup

Preference enumeration scales at 1980 → 579 ms (`#####`) and 838 → 238 ms (`####.####`) from one to eight threads: 3.4× and 3.5×. The gap from linear is memory-bound, not lock-bound — the loop body is a `pred_lexpos_` chase over two large arrays with no shared mutable state. The per-root DP parallelizes too, but at 15 ms of work there is nothing to win.

Absolute Phase II times vary by 10–20 % between runs on the same input; the phase is serial and cache-sensitive. The proportions in Table 5 are stable.

16 Where `tree-dp` stands

16.1 Small instances against the ILP optimum

`test_ilp_cases.sh` solves ten fixed instances exactly with CBC over the *full* link universe, so the optimum lower-bounds every heuristic. Re-run at `7f9127b`:

`tree-dp` attains the optimum on 7 of the 11 rows (the $(\texttt{AGCT})^5$ row was added for this note; the script's own ten give 6). Where it misses it misses by 1, 2, 1 and 5. Two of the four misses are the counterexamples of §12 and §13; the note-`##` row is missed by every heuristic, including `tree-dp3`.

16.2 The 139-configuration suite

`compare_algos` builds 139 synthetic repeat configurations — motif repeats, sparse insertions, mixed structures — and runs all five algorithms on each. Totals of $|C|$ over the whole suite, from the `7f9127b` build:

Table 6: Fixed ILP cases. Bold marks a heuristic short of the optimum.

instance	shape	opt	`greedy`	`dep-order`	`tree-dp`	`tree-dp2`	`tree-dp3`
note (n=40)	`#.#`	26	26	26	26	26	26
(`GCCTTTAAAG`)^3	`#.#`	14	14	14	14	14	14
(`AC`)^6	`#.#`	9	**10**	9	**10**	**10**	9
(`AGCT`)^5	`#.#`	11	**14**	11	11	11	11
(`ACGT`)^5	`#.#`	11	**14**	11	**13**	**13**	11
`GGGCGGCGGC`	`##`	11	11	11	11	11	11
note (n=40)	`##.##`	35	35	35	35	35	35
note (n=40)	`##`	19	**20**	**20**	**20**	**20**	**20**
`A`^{16}	`#.#`	11	11	11	11	11	11
(`ACGT`)^7	`##.#`	17	17	17	17	17	17
(`GCCTTTAAAG`)^4	`#..#`	22	**25**	**27**	**27**	**27**	22

Table 7: Suite totals, 139 configurations, no correctness failures.

$a_{\max}$	`greedy`	`dep-order`	`tree-dp`	`tree-dp2`	`tree-dp3`
8	259518	252357	252397	256197	252394
16	255785	252175	252329	255370	213078

Reading the default row ($a_{\max} = 8$):

- `tree-dp` beats `greedy` by 2.74 %. That is the headline number for the algorithm.
- `tree-dp` versus `tree-dp2` — the same algorithm with Phase II disabled — is 252397 against 256197. **Phase II is worth** 3800 **positions,** 1.5 %, more than the gap between `tree-dp` and `greedy` is worth over `dep-order`. The retarget loop is not a polish step.
- `tree-dp` versus `dep-order` is 252397 against 252357: `tree-dp` is 40 positions (0.016 %) *worse* suite-wide. The forest DP's global reasoning about hubs does not, on this suite, pay for the preference and cut-edge restrictions it imposes. Both algorithms share Phase II, so the comparison isolates Phase I, and Phase I is roughly a wash.
- `tree-dp3`, which re-solves clusters exactly on top of the same Phase II, lands at 252394 — three positions better than `tree-dp`. At $a_{\max} = 8$ the candidate universe, not the neighbourhood size, is what binds.

Reading the $a_{\max} = 16$ row: `tree-dp` barely moves (252397 → 252329, 0.03 %) while `tree-dp3` drops by 15.5 %. This is the quantitative form of the remark in §3: enlarging the candidate universe is worthless to an algorithm that only ever looks at one candidate per name.

16.3 Summary of the empirical picture

`tree-dp` is a solid, fast, deterministic heuristic that is optimal on most small instances and 2.7 % better than `greedy` on the suite. It is not better than `dep-order`, and most of what separates both from `greedy` is Phase II, which they share. The forest DP's contribution is real but small, and it is bounded above by the three restrictions of Fact 2 — which is what motivated `tree-dp3`.

17 Discussion

What the DP buys. The pseudoforest property (Corollary 2) is a real structural gift: it turns an NP-hard packing problem into a linear-time tree DP. The DP then answers a question no

greedy can: *is this hub worth more as a source than as a target?* On $(\texttt{AGCT})^5$ it answers correctly (6 against 9), keeps `AC` free, and reaches the optimum where `greedy` loses three positions.

What it costs. The gift is paid for by three restrictions, and §12–§13 show two of them binding on instances of thirteen and twenty-one ranks. The most uncomfortable is Fact 1: on a cycle, the algorithm's answer depends on the alphabetical order of k-mer names. It is deterministic — and the implementation goes to some trouble to keep it deterministic under parallelism — but determinism here means only that the same arbitrary choice is made every time.

An obvious improvement that is not obviously enough. Cutting the *minimum-coverage* edge of each cycle instead of the last-visited one would be a two-line change, and it would fix $(\texttt{AC})^6$: the cycle's two edges carry coverage 3 and 4, and dropping the cheaper one leaves exactly the orientation that reaches 9. It would *not* fix $(\texttt{ACGT})^5$, whose cycle carries coverages $4, 4, 3$ — the minimum-coverage edge is the one the current rule already discards, so the heuristic changes nothing there. That is the general problem with edge-local repairs: the value of an edge to the DP is not its coverage but the difference it makes to the subtree costs, which is not known before the DP runs.

The principled version — try all ν cuts of each cycle and keep the best — costs a factor ν on cycle components only, which is probably affordable. But it raises the question of what "best" means, and the measured answer is discouraging. The two forests of §12 report DP costs of 10 (the path) and 11 (the star): the path is the better forest *by the DP's own objective* and the worse one by two positions in the final answer, because the DP objective cannot see that the star leaves an interval unpinned for the leftover pass to exploit. Selecting cuts by DP value would have picked the wrong forest here. A correct rule would have to score each cut by running the rest of the pipeline, which is no longer a two-line change. `tree-dp3` instead lifts all three restrictions locally, after the fact, and gets 15.5 % at $a_{\max} = 16$.

Remark 5 (Measured, after this note was drafted). A variant implementing exactly the minimum-coverage repair (`tree-dp4`, added at `2995a80`, after the `7f9127b` snapshot this note documents; plain `tree-dp` is unchanged there) behaves precisely as the two paragraphs above predict, in both directions. On $(\texttt{AC})^6$ it returns the optimum 9 instead of 10. On the $(\texttt{ACGT})^5$ orbit it returns 13 on *all* 24 permutations — so it is now order-independent, which is progress, but it has given up the eight permutations on which plain `tree-dp` reached 11. Preferring the higher-coverage edge converts the star of Figure 2 into the path of Figure 3, which is the better forest by the DP's objective and the worse one by two positions after the leftover pass runs. Edge-local repair of the cut is a genuine improvement on some instances and a genuine regression on others, for the reason given above: coverage is not the quantity that matters.

The leftover pass deserves better billing. It is named as if it handled stragglers, but Fact 3 shows it is structurally responsible for *all* intra-interval links, and §11 shows it contributing 30 % of the compression on an instance where the DP itself did well. On $\texttt{A}^{16}$ it does everything. A reasonable reading of Algorithm 4 is that `tree-dp` has two heuristics, one exact-on-a-forest and one greedy, and that the second is not a fallback.

Where the time goes. Table 5 is unambiguous: on a gapped shape at 9 Mbp, Phase II is 97 % of the work and the entire forest machinery is 0.3 %. Any future effort spent making the DP faster is misdirected; any spent parallelizing or pruning the retarget loop is not.